

Exception Handling in Programming

Concepts and Code Examples

BAKD

DoCE

What is an exception?

*An *exception* is an error condition that changes the normal flow of control in a program

*When an Exception occurs the normal flow of the program is disrupted and the program/Application terminates abnormally, which is not recommended, therefore these exceptions are to be handled

Why Exception occurs?

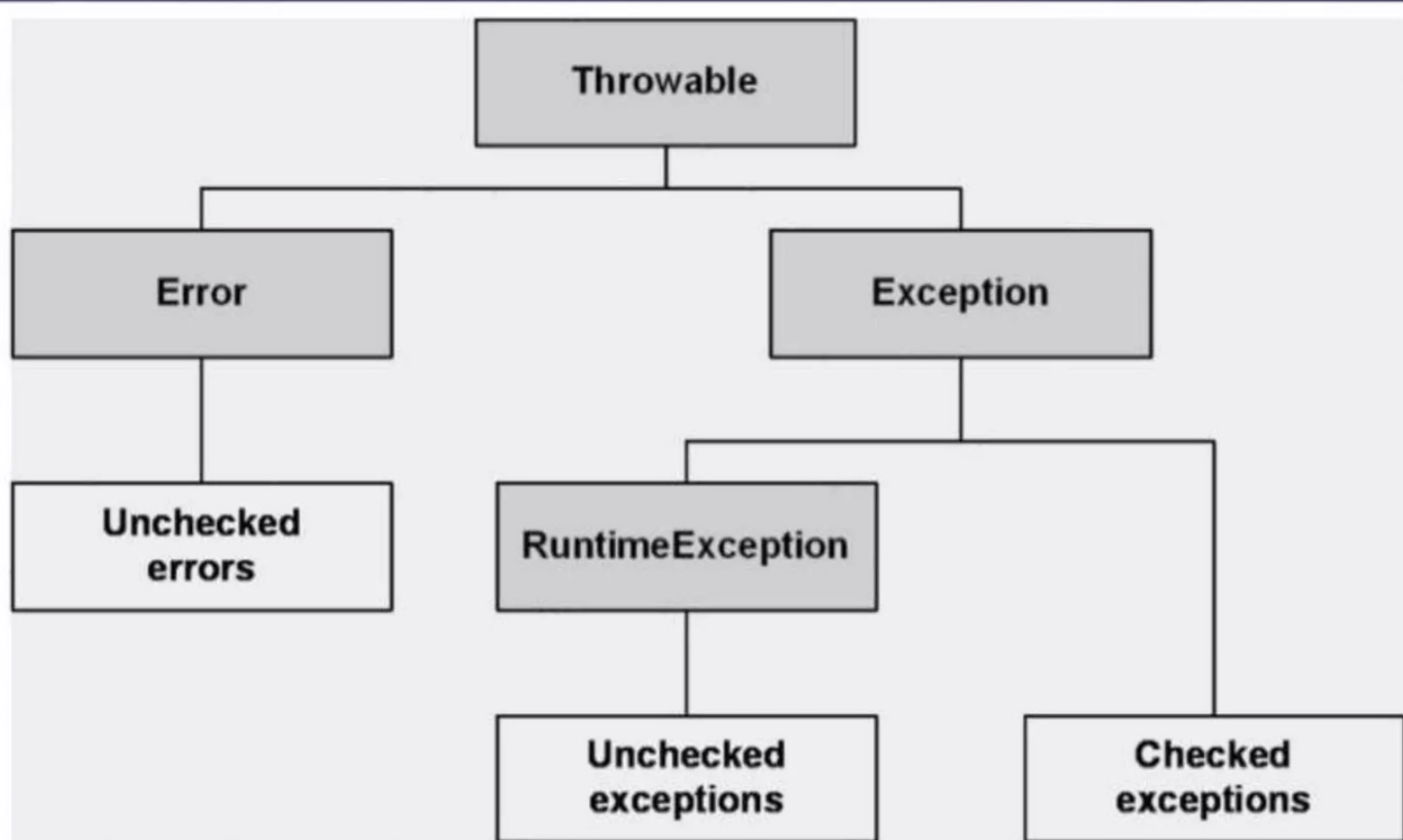
An exception is a problem that arises during the execution of a program. It can occur for various reasons say-

- A user has entered an invalid data
- File not found
- A network connection has been lost in the middle of communications
- The JVM has run out of a memory

Types of Exceptions

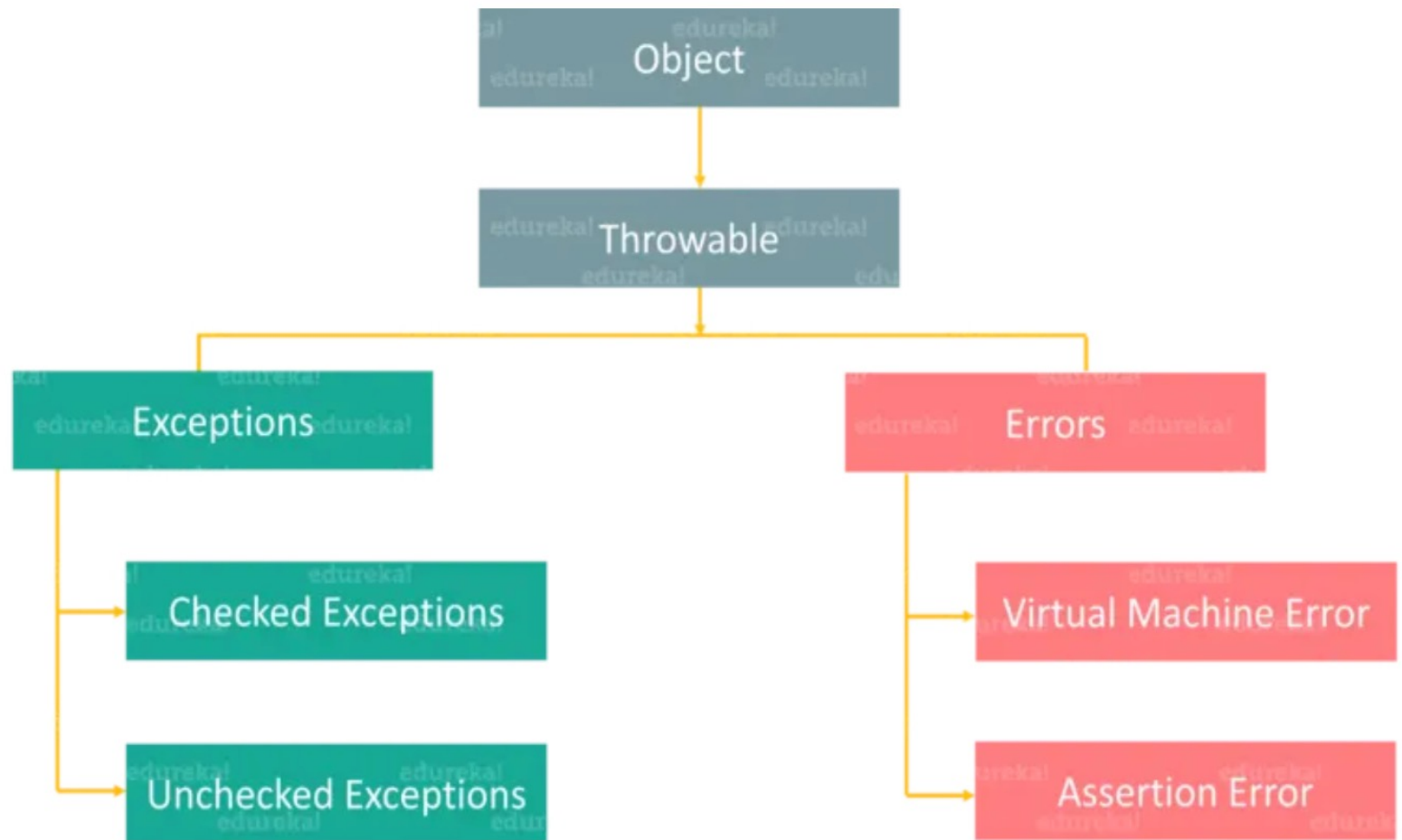
- Checked exceptions (compile-time)
- Unchecked exceptions (runtime)
- Error class for serious failures

Exception Hierarchy



Exceptions Hierarchy

All exception and error types are subclasses of class ***Throwable***, which is the base class of hierarchy. One branch is headed by ***Error*** which occurs at run-time and other by ***Exception*** that can happen either at compile time or run-time.



Exceptions Hierarchy - Java Exception Handling

- Basically, an Error is used by the Java run-time system (JVM) to indicate errors that are associated with the run-time environment (JRE). StackOverflowError is an example of such an error. Whereas Exception is used for exceptional conditions that user programs should catch. NullPointerException is an example of such an exception.

difference between errors and exceptions

Errors	Exceptions
1. Impossible to recover from an error	1. Possible to recover from exceptions
2. Errors are of type 'unchecked'	2. Exceptions can be either 'checked' or 'unchecked'
3. Occur at runtime	3. Can occur at compile time or run time
4. Caused by the application running environment	4. Caused by the application itself

Exception Has two Main classes :

1. **Checked exceptions** : known as compile time exceptions.

Programmer should take care of (handle) these exceptions

2. **Unchecked exceptions** : Known as Runtime Exceptions.

These include programming bugs, such as logic error also.

Basic Example of Exception

```
class Exception{  
    public static void main(String args[]){  
        try{  
            //code that may raise exception  
        }  
        catch(Exception e){  
            // rest of the program  
        }  
    }  
}
```

Above code represent an exception wherein inside try block.

try-catch-finally Example

```
try {  
    int a = 10 / 0;  
}  
catch (ArithmeticException e) {  
    System.out.println("Division by zero");  
}  
finally {  
    System.out.println("End of execution");  
}
```

Unchecked Exception Example

```
int[] arr = {1,2,3};  
System.out.println(arr[5]);
```

Exception Propagation

Exception flows
up the call stack

Handled by
nearest matching
catch block

Propagation Example

```
void risky() {  
    int x = 10 / 0;  
}  
  
public static void main(String[] args) {  
    try {  
        risky();  
    } catch (Exception e) {  
        System.out.println("Handled");  
    }  
}
```

Checked exceptions

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the Cloneable interface.
IllegalAccessException	Access to a class is denied.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.

Types of Exceptions

Built-in exceptions

Built-in Exceptions	Description
<i>ArithmeticException</i>	It is thrown when an exceptional condition has occurred in an arithmetic operation.
<i>ArrayIndexOutOfBoundsException</i>	It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.
<i>ClassNotFoundException</i>	This exception is raised when we try to access a class whose definition is not found.
<i>FileNotFoundException</i>	An exception that is raised when a file is not accessible or does not open.
<i>IOException</i>	It is thrown when an input-output operation is failed or interrupted.
<i>InterruptedException</i>	It is thrown when a thread is waiting, sleeping, or doing some processing, and it is interrupted.
<i>NoSuchFieldException</i>	It is thrown when a class does not contain the field (or variable) specified.

User-Defined Exceptions

Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, a user can also create exceptions which are called 'User-Defined Exceptions'.

```
class MyException extends Exception{
    String str1;
    MyException(String str2) {str1=str2;}
    public String toString(){
        return ("MyException Occurred: "+str1);
    }
}

class Example1{
    public static void main(String args[]){
        try{
            System.out.println("Start of try block");
            throw new MyException("Error Message");
        }
        catch(MyException exp){System.out.println("Catch Block");
            System.out.println(exp);
        }
    }
}
```

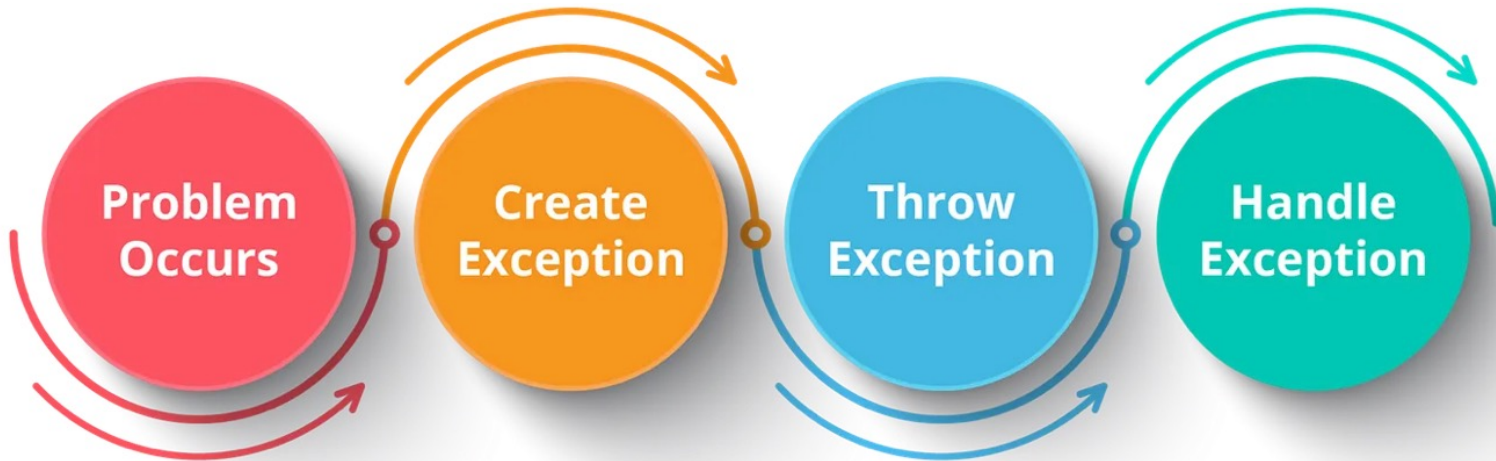


What is Exception Handling?

- Handles runtime errors
- Prevents abnormal program termination
- Improves program robustness

Exception handling is a programming mechanism to manage unexpected errors (exceptions) during program execution

Exception Handling mechanism follows a flow which is depicted in the below figure. But if an exception is not handled, it may lead to a system failure. That is why handling an exception is very important.



The flow of Exception Handling - Java Exception Handling



Exception Handling Blocks

- try – code that may cause exception
- catch – handles exception
- finally – always executes

Exception Handling – Handler Terminology

Try Block

- **The code region where exceptions may be generated and are monitored for handling.**

- **Catch Block / Except Block**

The block that **receives and handles a specific type of exception.**

- catch → Java, C++
- except → Python

Finally Block

A block that **always executes** after try/catch, whether an exception occurs or not.

Used for:

- Resource cleanup
- Closing files
- Releasing locks

Multiple Catch Handlers

Using several catch blocks to handle different exception types separately.

Catch-All Handler

A handler that catches any exception type.

Example:

```
catch(Exception e)
```

Cleanup Handler

A handler (often finally) whose main role is resource deallocation and cleanup.

1. try: A block of code where potential errors are tested.
2. catch: A block of code that executes if an exception occurs in the try block, handling the error.
3. throw: A statement used to explicitly create and trigger (throw) an exception.
4. throws: A keyword in method declarations indicating that the method might throw specific exceptions, passing responsibility to the caller.
5. finally: A block that runs after try and catch blocks, used for cleanup (like closing files), regardless of whether an exception occurred.
6. else: (Python) A block that runs only if the try block completes without any exceptions.

Language-wise Exception Handling Summary Chart comparing C++ / Java / Python, suitable for final-year Computer Engineering students and very useful for exams and revision.

Exception Handling: C++ vs Java vs Python



Aspect	C++	Java	Python
Keyword to monitor code	try	try	try
Keyword to handle exception	catch	catch	except
Keyword to throw exception	throw	throw	raise
Finally block	Not built-in (catch + destructors used)	finally supported	finally supported
Checked exceptions	✗ Not supported	✓ Supported	✗ Not supported
Unchecked exceptions	✓ All exceptions	✓ Runtime exceptions	✓ All exceptions
Exception base class	<u>std::exception</u>	Throwable → Exception	BaseException → Exception
Multiple handlers	Multiple catch blocks	Multiple catch blocks	Multiple except blocks
Catch-all handler	catch(...)	<u>catch</u> (Exception e)	except Exception:
Custom exceptions	Inherit from <u>std::exception</u>	Extend Exception class	Inherit from Exception
Rethrow exception	throw;	throw e; or throw;	raise
Resource management	RAII, destructors	finally, try-with-resources	with statement, finally
Try-with-resources / auto close	✗ (RAII instead)	✓ try-with-resources	✓ with statement
Exception specification	Deprecated (<u>throw()</u>)	throws clause required for checked exceptions	✗ No declaration needed
Termination model	Yes	Yes	Yes
Resumption model	✗ Not supported	✗ Not supported	✗ Not supported
Stack unwinding support	✓ Automatic	✓ Automatic	✓ Automatic
Built-in common exceptions	bad_alloc, out_of_range	NullPointerException, IOException	ValueError, TypeError
Assertion vs Exception	assert macro	assert keyword	assert statement